

Simulation Exercise

Creating a Personal Financial Planning Web App

AI for Financial Services: Tools, Opportunities, and Challenges
July 24, 2026

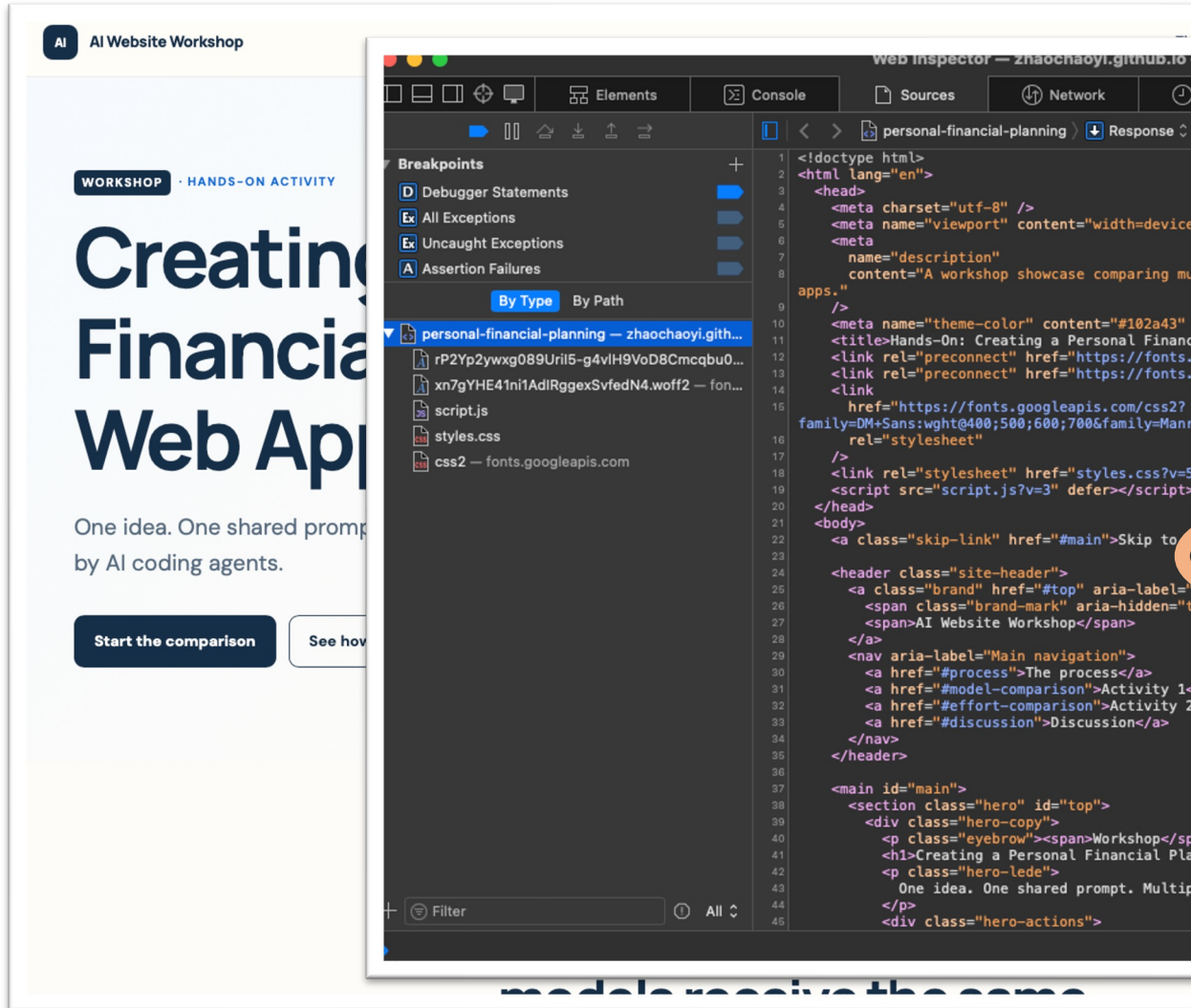
Chaoyi Zhao

Postdoctoral Associate, MIT Sloan School of Management and Laboratory for Financial Engineering

Many personal financial planning websites



Workshop website (by ChatGPT 5.6 Sol, medium effort)

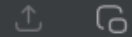


Andrew's (verbal) prompt

Could you show the audience how AI can be used to build a personal financial planning website and publish it through GitHub so that it is publicly accessible, thereby demonstrating the capabilities of AI?

My prompt

ChatGPT 5.6 Thinking >



I want to use Codex to build a small educational web app for a workshop. The app will be a Personal Financial Planning Scenario Planner. It will demonstrate how an AI coding agent can turn an idea into a working website. I have no technical background, and I want to publish the website publicly using GitHub. What should my first prompt be?

A strong first prompt should define the audience, features, design constraints, and deployment target while letting Codex make sensible technical choices. Codex can work from a local folder or Git repository and iterate on the result; OpenAI still recommends reviewing and testing agent-generated code before publishing it. [OpenAI +1](#)

Paste this into Codex:

Writing



Build a complete, beginner-friendly educational web app called **Personal Financial Planning Scenario Planner**.

Purpose

This app will be demonstrated in a workshop to show how an AI coding agent can turn a plain-language idea into a working public website.

I have no technical background, so please make sensible decisions for me, explain important choices in plain English, and avoid unnecessary complexity.

Technical requirements



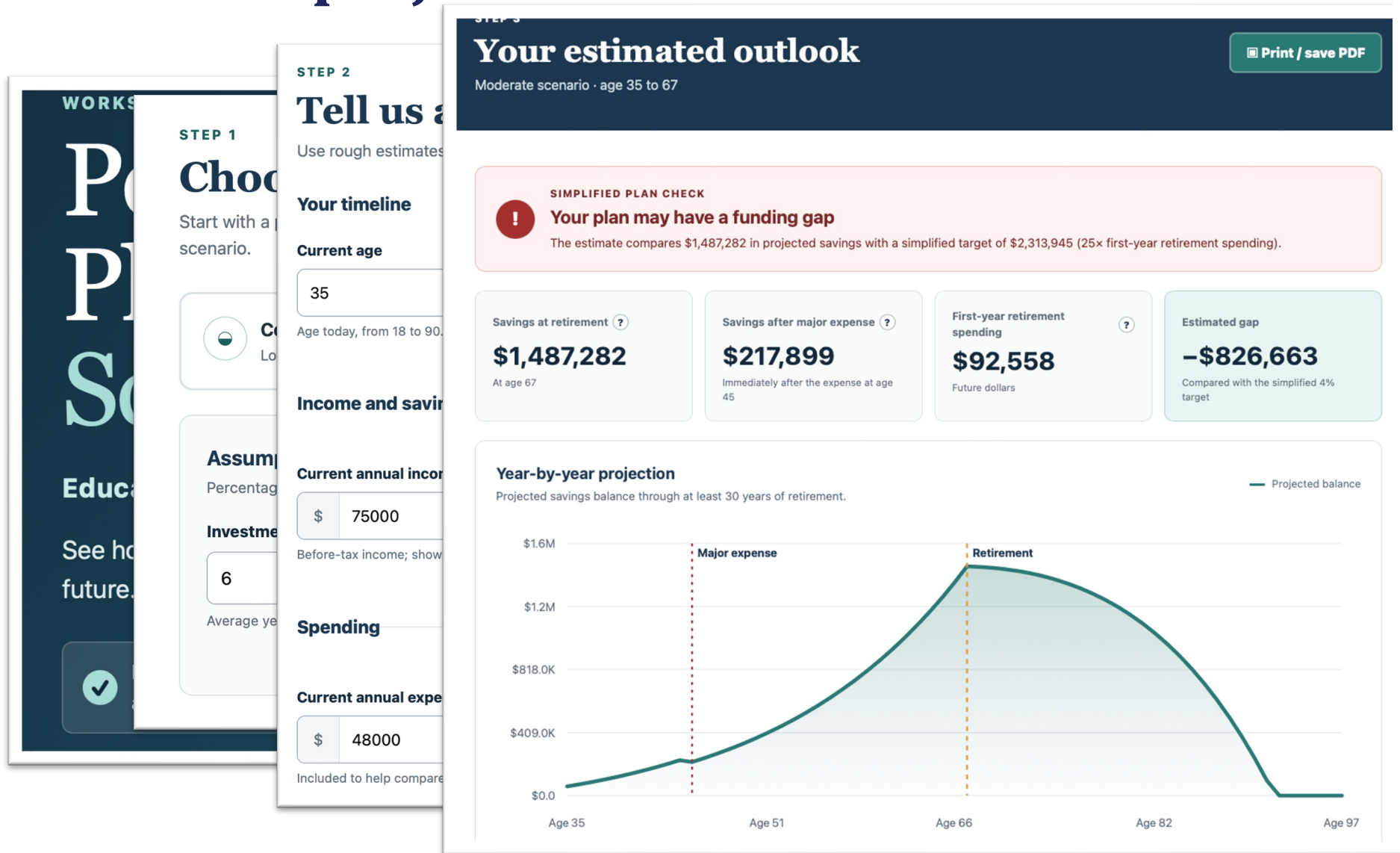
- Build a static website that can be hosted free on **GitHub Pages**.

Ask anything

+ 🌐 🗨️ 5.6 Thinking



The demo project: a Financial Scenario Planner



How AI contributed to this exercise



The prompt

Written by ChatGPT 5.6 Sol—I asked the AI what I should ask the coding agent. (Meta-prompting.)



The web app

Built end-to-end by Codex and Claude Code from that single plain-English prompt.



The publishing

GitHub repository and GitHub Pages setup.



These slides

First draft generated by Claude Code (Fable 5).

- *No software engineering background is needed.*
- *Fluency in describing problems is becoming as valuable as fluency in code.*

Plain language is the new programming interface



0

lines of code written by hand

\$0

hosting cost on GitHub Pages

min

delivery measured in minutes,
not weeks

The skill being demonstrated is not coding—it is specifying, directing, and verifying.

PART 1

The Workflow

Step 0 — One-time setup (before the magic)



Install VS Code

Free code editor from Microsoft. You will mostly watch, not type code.



Add the agent extensions

Install the Codex extension (OpenAI) and/or the Claude Code extension (Anthropic) from the VS Code marketplace.



Sign in

Codex uses your ChatGPT account; Claude Code uses your Claude account. Paid plans include agent usage.



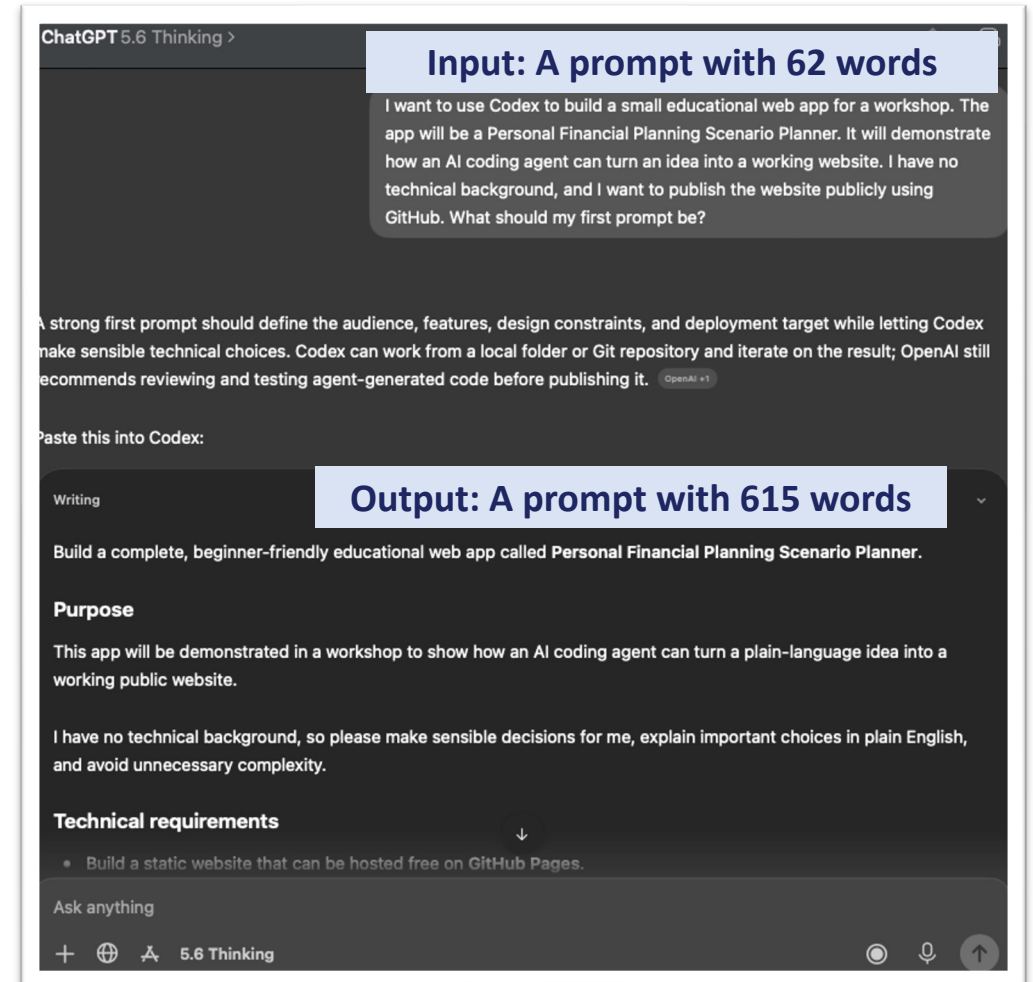
Create an empty folder

Make a new folder, open it in VS Code.

Step 1 — Ask the AI what to ask the AI

Instead of guessing what a coding agent needs to hear, I asked GPT to write the prompt for me:

“I want to use Codex to build a small educational web app for a workshop. The app will be a Personal Financial Planning Scenario Planner. It will demonstrate how an AI coding agent can turn an idea into a working website. I have no technical background, and I want to publish the website publicly using GitHub. What should my first prompt be?”



What GPT wrote back

A strong first prompt should define the audience, features, design constraints, and deployment target while letting Codex make sensible technical choices. Codex can work from a local folder or Git repository and iterate on the result; OpenAI still recommends reviewing and testing agent-generated code before publishing it.

Paste this into Codex:

Build a complete, beginner-friendly educational web app called Personal Financial Planning Scenario Planner.

Purpose

This app will be demonstrated in a workshop to show how an AI coding agent can turn a plain-language idea into a working public website.

I have no technical background, so please make sensible decisions for me, explain important choices in plain English, and avoid unnecessary complexity.

What GPT wrote back

Technical requirements

- * Build a static website that can be hosted free on GitHub Pages.
- * Use plain HTML, CSS, and JavaScript unless there is a compelling reason to use something more complex.
- * Do not require a backend, database, login, paid service, API key, or build server.
- * Store no personal data.
- * Make the site responsive and accessible.
- * Use clean, well-organized, commented code.
- * Create all necessary project files, including:
 - * index.html
 - * CSS and JavaScript files
 - * README.md
 - * .gitignore
 - * a suitable open-source license
- * Make sure the site works when opened locally and when deployed from a GitHub Pages repository.

What GPT wrote back

App experience

Create a polished single-page scenario planner that helps a user explore a simplified personal financial plan.

Include input sections for:

1. Current age
2. Planned retirement age

...

Include three selectable scenarios:

- * Conservative
- * Moderate
- * Optimistic

Each scenario should adjust assumptions such as investment return, income growth, and inflation. Clearly show the assumptions being used and allow users to customize them.

What GPT wrote back

Results

As the user changes inputs, calculate and display:

- * Estimated savings at retirement
- * Estimated savings after the major future expense
- * Estimated first-year retirement spending
- * A simple estimate of whether the plan appears on track
- * The estimated gap or surplus
- * A year-by-year projection chart
- * A clear comparison of the three scenarios

Use transparent, simplified calculations. Add tooltips or short explanations for financial terms and calculation assumptions.

Include a prominent disclaimer explaining that this is an educational demonstration and not financial, tax, investment, or legal advice.

What GPT wrote back

Design

Use a welcoming, professional workshop-friendly design.

- * Clear headings and generous spacing
- * Easy-to-read typography
- * Accessible form labels

...

Include example starting values so the app is useful immediately when opened.

Add buttons to:

- * Reset the planner
- * Load an example scenario
- * Print or save the results as a PDF using the browser's print feature

What GPT wrote back

Quality checks

Before finishing:

1. Test the calculations with several example values.
2. Check for JavaScript errors.
3. Check the mobile layout.
4. Check keyboard accessibility.
5. Confirm that no sensitive data is transmitted or stored.
6. Confirm that all links and buttons work.
7. Confirm that the project is compatible with GitHub Pages.

What GPT wrote back

Documentation and publishing

Write a beginner-friendly README.md that explains:

- * What the app does
- * The project file structure
- * How to preview it locally

...

At the end of your work, give me:

1. A plain-English summary of what you built
2. A list of the files you created

...

Start by briefly outlining your implementation plan, and then create the complete app. Do not stop after giving instructions or sample code—create all required files and test the implementation.

What GPT wrote back

Before sending the prompt, open Codex and select a new empty folder or Git repository for the project.

One useful workshop technique is to treat this as the first iteration, not the final specification. After Codex builds it, your next prompt can be: “Run the app, inspect it as a first-time workshop participant, fix anything confusing or broken, and make the financial assumptions easier to understand.”

Step 2 — Paste the prompt, watch the agent work

Worked for 11m 23s ▾

I'll build this as a small, dependency-free website so it opens directly from a folder and deploys unchanged to GitHub Pages. I'll first inspect the workspace, then create the page, styling, calculation logic, tests/documentation, and finally run automated checks plus a local browser-style smoke test where the available tools allow it.

The workspace is empty, so there's no existing project to preserve. I'm using a transparent model: savings grow annually, contributions rise with income growth, a future expense is deducted in the chosen year, and retirement readiness is compared with a simplified 4% spending target. The app will state these assumptions plainly and keep every calculation in the browser.

🔗 Edited files ▾

🔗 Created file ▾

```
app.js +246 -0
```

```
3
4   const presets = {
5     conservative: { label: "Conservative", investmentReturn: 4, incomeGrowth: 1.5, inflationRate: 3.5 },
6     moderate: { label: "Moderate", investmentReturn: 6, incomeGrowth: 2.5, inflationRate: 2.5 },
7     optimistic: { label: "Optimistic", investmentReturn: 8, incomeGrowth: 3.5, inflationRate: 2 }
8   };
9   const scenarios = JSON.parse(JSON.stringify(presets));
10  const exampleValues = {
11    currentAge: 35, retirementAge: 67, annualIncome: 75000, currentSavings: 60000,
```

The complete app and beginner documentation are now in place. I've also separated the financial math from the page interactions, which makes the assumptions easier to audit and lets the core calculations be tested automatically without adding any framework or dependency. Next I'm checking syntax, calculations, links, controls, and responsive behavior.

The core calculation tests pass, and the JavaScript files have no syntax errors. The privacy scan also found no storage, cookies, analytics, or network calls. During review I found two small edge cases worth tightening: an expense scheduled for the user's current age should be deducted immediately, and the chart should extend far enough to include a later selected expense. I'm correcting those before the final audit.

The page now renders correctly in Firefox at both desktop and phone widths; the phone layout stacks cleanly with readable spacing, and the live JavaScript initializes the selected scenario as expected. I'm doing the final edge-case assertions and removing the temporary screenshots used for visual QA so only real project files remain.

Complete

Built a polished, responsive Personal Financial Planning Scenario Planner that runs entirely in the browser and is ready for GitHub Pages.

It includes:

↓

Live demo

Survey: Which is the best app?



https://docs.google.com/forms/d/e/1FAIpQLSeHDOeoGhhmf31fN_gXTILoPl1Aj-6ZUngaoDJ5NW_asrTZmw/viewform

1. Which website do you like the **most**? *

- ☐ C - <https://zhaochaoyi.github.io/personal-financial-planning-c1/>
- ☐ E - <https://zhaochaoyi.github.io/personal-financial-planning-e1/>
- ☐ G - <https://zhaochaoyi.github.io/personal-financial-planning-g1/>

2. Which website do you like the **least**? *

- ☐ C - <https://zhaochaoyi.github.io/personal-financial-planning-c1/>
- ☐ E - <https://zhaochaoyi.github.io/personal-financial-planning-e1/>
- ☐ G - <https://zhaochaoyi.github.io/personal-financial-planning-g1/>

PART 2

Same Prompt, Different Models

A controlled experiment

Same prompt · same empty folder · same instructions.



Runtime

Wall-clock time from prompt to finished project.



Tokens & cost

How much “thinking” each model spent.



Design taste

Layout, typography, color choices—models have styles.



Correctness

Do the compound-interest projections actually check out?



Spec compliance

Did it deliver every requirement?



Documentation

Is the README truly beginner-friendly?

Survey results



https://docs.google.com/forms/d/e/1FAIpQLSeHD OeoGhhmf31fN_gXTILoPI1Aj-6ZUngaoDJ5NW_asrTZmw/viewanalytics

Model	Effort	Mapping
GPT-5.6 Sol	Medium	C
Claude Fable 5	Medium	E
Claude Sonnet 5	Medium	G

My results

Model	Effort	Time	Usage (weekly limit)	Mapping	Live site
GPT-5.6 Sol	Medium	11m 23s	5%	C	https://zhaochaoyi.github.io/personal-financial-planning-c1/
GPT-5.5	Medium	6m 34s	2%	F	https://zhaochaoyi.github.io/personal-financial-planning-f1/
GPT-5.4	Medium	6m 39s	2%	B	https://zhaochaoyi.github.io/personal-financial-planning-b1/
Claude Fable 5	Medium	-	5%	E	https://zhaochaoyi.github.io/personal-financial-planning-e1/
Claude Opus 4.8	Medium	-	3%	D	https://zhaochaoyi.github.io/personal-financial-planning-d1/
Claude Sonnet 5	Medium	-	2%	G	https://zhaochaoyi.github.io/personal-financial-planning-g1/
Claude Haiku 4.5	-	-	1%	A	https://zhaochaoyi.github.io/personal-financial-planning-a1/

Note: Results vary run to run and versions update. Claude Code does not have a clock. Haiku does not have an effort option.

My results

Model	Effort	Time	Usage (weekly limit)	Mapping	Live site
GPT 5.6 Sol	Ultra	35m 40s	12%	5	https://zhaochaoyi.github.io/personal-financial-planning-c5/
GPT 5.6 Sol	Extra High	23m 8s	8%	2	https://zhaochaoyi.github.io/personal-financial-planning-c2/
GPT 5.6 Sol	High	15m 18s	5%	3	https://zhaochaoyi.github.io/personal-financial-planning-c3/
GPT 5.6 Sol	Medium	11m 23s	5%	1	https://zhaochaoyi.github.io/personal-financial-planning-c1/
GPT 5.6 Sol	Light	5m 46s	2%	4	https://zhaochaoyi.github.io/personal-financial-planning-c4/

Note: Results vary run to run and versions update.

Patterns



Every frontier model clears the bar

All of them produce a working site from this prompt.
The differences are in polish, structure, speed, and cost; not in “can it do it.”



Newer ≠ uniformly better

A newer version may be slower or spend more tokens for a similar result.
Match the tool to the task, not to the release date.



Models have taste

Same spec, visibly different design decisions—color, layout, wording.



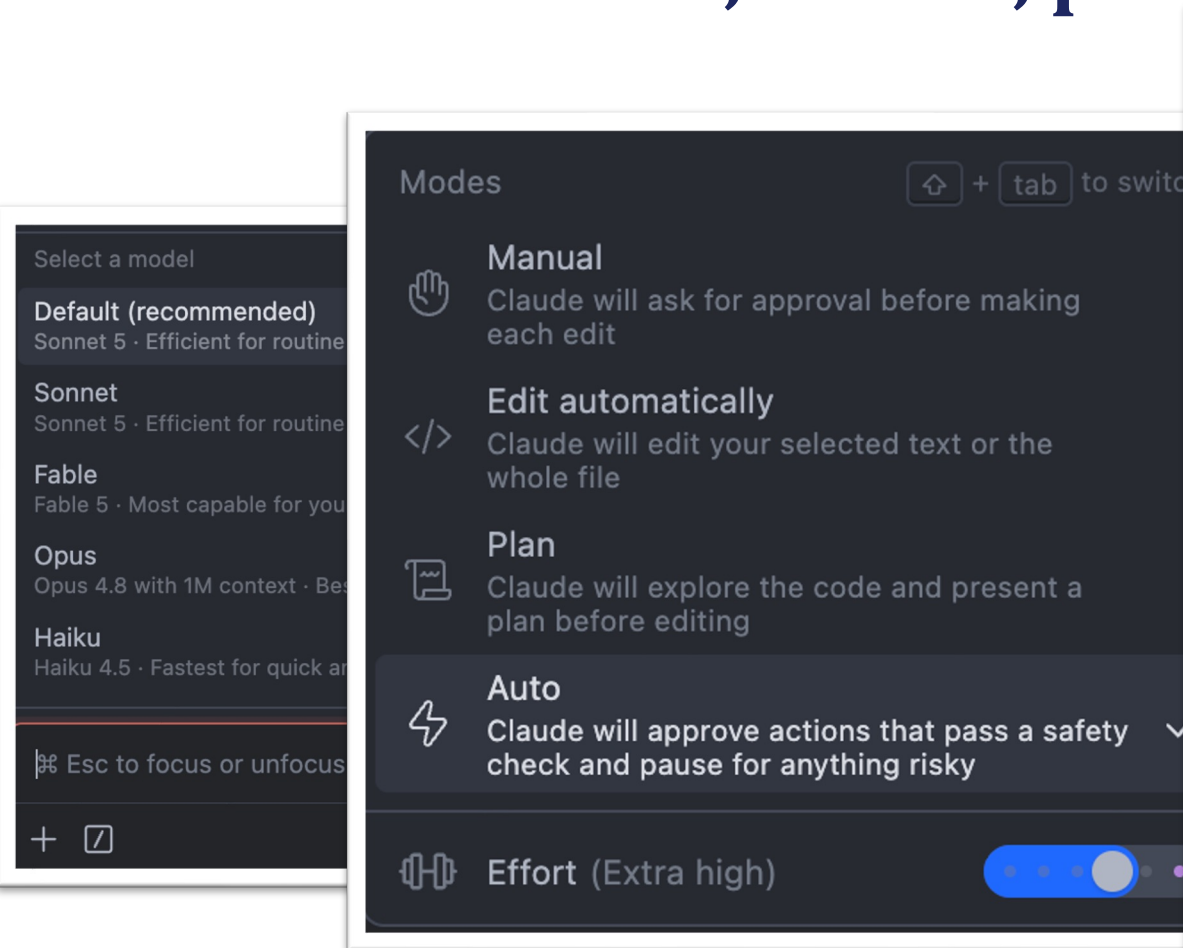
Cost scales with “thinking”

Higher effort settings and bigger models buy depth with time and tokens.
For a one-page static site, medium effort is usually plenty.

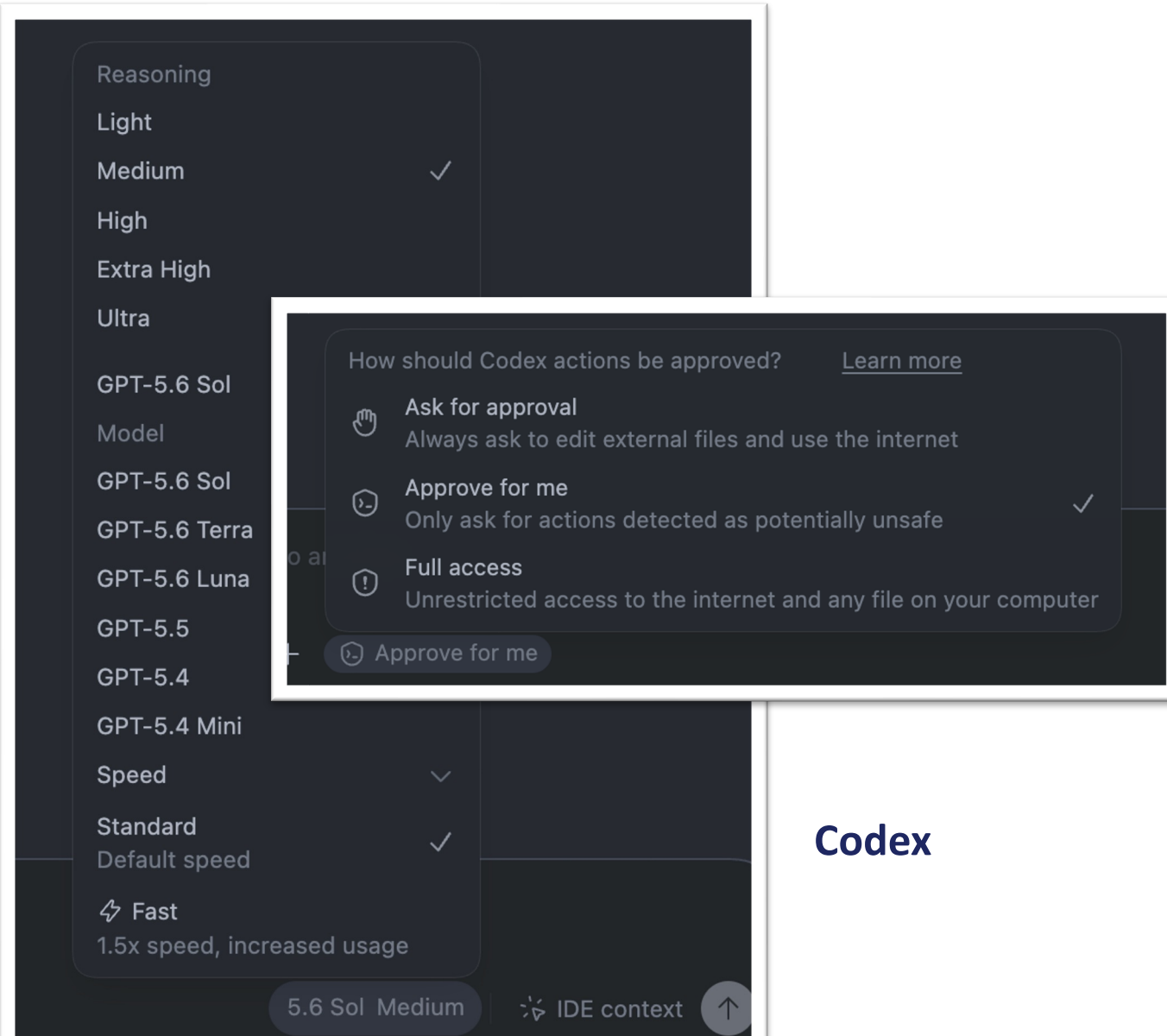
PART 3

Choices That Matter

Choices: models, efforts, permissions



Claude Code



Codex

Reasoning effort: how hard should it think?

Both Codex and Claude Code let you dial how much internal reasoning the model does before acting. It is a three-way trade between depth, speed, and cost.

Low

Fast & cheap

- Small, well-defined edits
- Renaming, formatting, tiny fixes
- Quick questions about the code

Medium

The default

- Building this workshop app
- Typical bug fixes
- Most day-to-day work

High

Slow & thorough

- Architecture decisions
- Confusing bugs
- Security- or correctness-critical code

Permissions: auto-approve or review every action?

Agents do real things: create and delete files, run commands, push to GitHub.
You choose how much they may do without asking.



Approve each step

The agent pauses before every file change and command; you click approve.

- See exactly what an agent does
- Safest with real data or shared machines
- Slower; approval fatigue is real



Middle ground

Auto-approve safe actions (reading files, edits inside the project folder); ask for the rest.

- Good pace without giving up oversight
- Commands, deletions, network access still need you
- The default choice for most real work



Full auto

The agent runs everything without asking.

- Riskier: it can run any command it decides on
- Never with credentials or real data

Trust, but verify



“Looks right” ≠ “is right”

AI is confident even when wrong.



Hallucination is real

Models can invent facts, library functions, and citations. Anything factual that matters must be checked against a primary source.



Privacy & secrets

Never paste passwords, API keys, or personal data into prompts.



Licenses & disclaimers

Keep the “educational demo, not financial advice” disclaimer prominent.

PART 4

AI in Practice

The skill stack is shifting

From → To

Writing every line → **Specifying & directing the work**

Memorizing syntax → **Decomposing problems clearly**

Debugging alone → **Evaluating & steering an agent**

“Does it run?” → **“Is it correct, safe, and honest?”**



Specification

Turning fuzzy goals into precise, constrained requests.



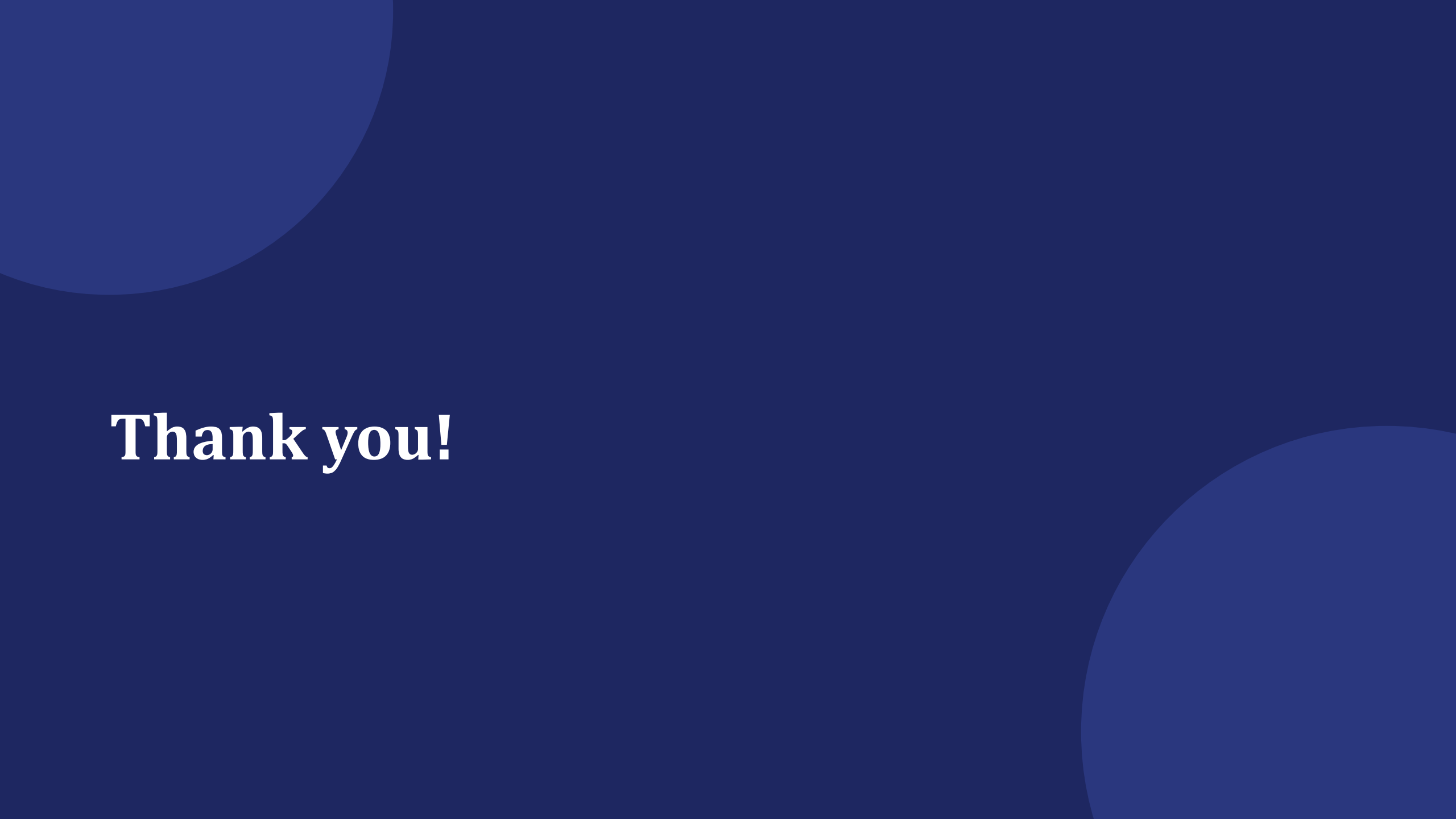
Verification

Verify results before approving.



Judgment & ethics

When to trust, when to escalate, who is accountable, what data may be shared.



Thank you!